

# API

Every route is reached through `api-gateway`. The frontend never calls a service directly, and no service is published outside the compose network.

Base URL: `https://<host>/api` (the gateway is `:8080` when run directly).

## Conventions

- Request and response bodies are JSON, except the upload endpoint.
- Errors are `{ "error": "message" }` with a meaningful status. Validation failures may add `details`.
- Authentication is a session cookie set by `/api/auth/sign-in/email`.
- `401` means not signed in; `403` means signed in but not allowed.

The gateway verifies the session once and forwards identity downstream as `x-skillforge-user-id`, `-email` and `-role`, signed with `x-skillforge-gateway-key`. Those headers are stripped from inbound requests — a client cannot set them.

## auth-service — `/api/auth/*`

Better Auth owns this prefix. The routes below are the ones the frontend uses.

| Method | Path                                 | Body                                   | Returns                                                  |
|--------|--------------------------------------|----------------------------------------|----------------------------------------------------------|
| POST   | <code>/api/auth/sign-up/email</code> | <code>{ email, password, name }</code> | <code>{ token, user }</code> and sets the session cookie |
| POST   | <code>/api/auth/sign-in/email</code> | <code>{ email, password }</code>       | <code>{ token, user }</code> and sets the session cookie |
| POST   | <code>/api/auth/sign-out</code>      | <code>{}</code>                        | <code>{ success: true }</code>                           |
| GET    | <code>/api/auth/get-session</code>   | —                                      | the session, or <code>null</code>                        |

Passwords are at least 12 characters and hashed with argon2id at the OWASP baseline (m=19456, t=2, p=1). The hash lives in `accounts.password`; there is no `passwordHash` column on `users`.

Sign-in is rate limited to 5 attempts per 15 minutes per client.

## Roles

`role` is server-owned — `input: false` on the field, so a crafted signup body cannot escalate. Changing one is an admin action.

| Method | Path                      | Body     | Notes                                                                   |
|--------|---------------------------|----------|-------------------------------------------------------------------------|
| GET    | /api/auth/roles/users     | —        | admin only                                                              |
| PATCH  | /api/auth/roles/users/:id | { role } | admin only; student , mentor or admin . Refuses to change your own role |

## profile-api — /api/profile/\*

---

All routes require a session.

| Method | Path                            | Body                                                                      | Returns                                                                 |
|--------|---------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------|
| GET    | /api/profile/me                 | —                                                                         | { profile }                                                             |
| PUT    | /api/profile/me                 | { name?, headline?, bio?, education?, experienceLevel?, targetRoleSlug? } | { profile }                                                             |
| GET    | /api/profile/skills             | —                                                                         | { skills }                                                              |
| PUT    | /api/profile/skills             | { skills: [{ slug, level, evidence? }] }                                  | { skills }                                                              |
| DELETE | /api/profile/skills/:slug       | —                                                                         | { skills }                                                              |
| GET    | /api/profile/projects           | —                                                                         | { projects }                                                            |
| POST   | /api/profile/projects           | { title, description?, url?, startedAt?, completedAt?, skillSlugs? }      | { projects }                                                            |
| DELETE | /api/profile/projects/:id       | —                                                                         | { projects }                                                            |
| GET    | /api/profile/certifications     | —                                                                         | { certifications }                                                      |
| POST   | /api/profile/certifications     | { name, issuer?, issuedAt?, credentialUrl? }                              | { certifications }                                                      |
| DELETE | /api/profile/certifications/:id | —                                                                         | { certifications }                                                      |
| POST   | /api/profile/uploads            | multipart: file, optional purpose=cv                                      | { upload }                                                              |
| GET    | /api/profile/uploads            | —                                                                         | { uploads }                                                             |
| GET    | /api/profile/uploads/:id        | —                                                                         | the file, Content-Disposition: attachment                               |
| DELETE | /api/profile/uploads/:id        | —                                                                         | { ok: true }                                                            |
| GET    | /api/profile/students/:userId   | —                                                                         | { profile, skills }<br>— self, admin, or a mentor paired in mentorships |

#### Notes:

- PUT /skills writes source: "self\_reported" regardless of what is sent. A typed claim must never reach the gap calculation carrying an assessment's weight.
- Levels are 1–5, the same scale as role\_requirements, so comparing them is a subtraction. Only leaf skills (altitude = 'SKILL') accept a level.
- Uploads are limited to 5MB and to application/pdf, image/png, image/jpeg. The stored filename is generated, never the client's.

## Taxonomy — public

| Method | Path                          | Returns                                                                                                                                                                                           |
|--------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | /api/skills/taxonomy          | categories → subcategories → skills                                                                                                                                                               |
| GET    | /api/skills/roles             | every target role                                                                                                                                                                                 |
| GET    | /api/skills/graph?role=<slug> | the role's required subgraph plus transitive prerequisites, each node tagged <code>mastered</code> / <code>progress</code> / <code>gap</code> / <code>locked</code> , plus <code>readiness</code> |

/graph uses the caller's demonstrated levels when signed in, and an empty set otherwise — which renders as an all-gaps preview.

## Assessments

| Method | Path                                   | Body                                    | Returns                                                                          |
|--------|----------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------|
| GET    | /api/skills/assessments                | —                                       | published assessments with question counts and the caller's best attempt         |
| GET    | /api/skills/assessments/:slug          | —                                       | { assessment, questions } — <b>never</b> includes answer, correct or explanation |
| POST   | /api/skills/assessments/:slug/attempts | { }                                     | { attempt: { id, startedAt } }                                                   |
| POST   | /api/skills/attempts/:id/submit        | { answers: [{ questionId, response }] } | { result }                                                                       |
| GET    | /api/skills/attempts/:id               | —                                       | { result } with answers, explanations and the per-skill breakdown                |
| GET    | /api/skills/attempts                   | —                                       | the caller's recent attempts                                                     |
| GET    | /api/skills/students/:userId/attempts  | —                                       | one student's attempts, plus the per-skill breakdown of the most recent          |

/students/:userId/attempts is the mentor and admin read path. Authorization is `canReadStudent` in `packages/db` — the same `mentorships` join profile-api enforces over the profile itself, not a check that the caller's role string says "mentor". It returns scores and the breakdown, never the answers: a mentor needs the shape of a result, the paper belongs to the student.

`response` is a string for every question type. MCQ answers are comma-separated choice indices ( `"2"` , or `"0,3"` when more than one is correct).

Submitting is one transaction and does four things:

1. Grades every question in the bank — an unanswered question is wrong, so leaving it out cannot inflate the score.
2. Groups results by the skill each question is tagged with, producing a per-skill breakdown rather than one number.
3. Writes `student_skills` at `level = round(1 + ratio × 4)` , `source: "assessment"` . Rows sourced from a project or a certification are left alone — those were earned somewhere a quiz cannot see.
4. Advances the FSRS schedule for each skill, grading `1 – 4` from the ratio. A skill evidenced only by multiple choice gets a 0.7× interval, because recognising an answer is weaker evidence than producing one.

## Progress (FSRS)

| Method | Path                                           | Body                            | Returns                                       |
|--------|------------------------------------------------|---------------------------------|-----------------------------------------------|
| GET    | <code>/api/skills/progress</code>              | —                               | tracked skills, due count and mastery buckets |
| GET    | <code>/api/skills/progress/due</code>          | —                               | skills due now                                |
| POST   | <code>/api/skills/progress/:slug/review</code> | <code>{ grade }</code><br>(1–4) | the next due date and interval                |
| GET    | <code>/api/skills/progress/history</code>      | —                               | reviews per day for 90 days                   |
| GET    | <code>/api/skills/progress/reviews</code>      | —                               | recent reviews                                |

State is keyed on `(userId, skillId)` , not on a card. Proficiency decays and has to be re-earned, so "mastered" still means something a month later. A manual review moves the schedule but deliberately does not write `student_skills` — a self-graded review is not evidence of a level.

## Roadmap

| Method | Path                                 | Body                       | Returns                                  |
|--------|--------------------------------------|----------------------------|------------------------------------------|
| POST   | <code>/api/skills/roadmap</code>     | <code>{ roleSlug? }</code> | <code>{ roadmap, source }</code>         |
| GET    | <code>/api/skills/roadmap</code>     | —                          | the latest roadmap, or <code>null</code> |
| GET    | <code>/api/skills/roadmap/:id</code> | —                          | one roadmap                              |

`roleSlug` falls back to the profile's target role. `source` is `"python-analyzer"` or `"local"` . Generating archives the previous active roadmap rather than deleting it, so regenerating shows movement instead of overwriting the evidence of it.

Once the phases are settled, skill-service posts them to ai-service `/narrate` and stores what comes back in `roadmaps.narration` and `roadmap_phases.rationale` — the response reports it as `narrated`:

`true`. The call is bounded at 20 seconds and fails soft: an unreachable or unconfigured ai-service leaves both columns null and changes nothing else about the plan. Ordering is never a model's to decide; see the roadmap rules in `CLAUDE.md`.

## ai-service

Two prefixes reach the same service, and the difference matters.

- `/ai/*` — Next route handlers. They assemble the signed-in student's context server-side, then call ai-service. **This is what the UI calls.** The browser never states what a student has demonstrated.
- `/api/ai/*` — the gateway forwarding to ai-service unchanged. Raw service access, used for testing.

### What the UI calls

| Method | Path                    | Body                                         | Returns                          |
|--------|-------------------------|----------------------------------------------|----------------------------------|
| POST   | <code>/ai/chat</code>   | <code>{ question, history? }</code>          | <code>{ answer, sources }</code> |
| POST   | <code>/ai/agent</code>  | <code>{ question, history? }</code>          | <code>{ answer, steps }</code>   |
| POST   | <code>/ai/expand</code> | <code>{ skill, subcategory?, count? }</code> | <code>{ skills }</code>          |

`history` is the last 10 text turns as `{ role, content }`, trimmed client-side and re-validated server-side. Tool-call frames never leave the server.

`sources` is `[{ source, relevance }]` — the retrieved chunks, in citation order, so `[1]` in the answer is `sources[0]`. `steps` is `[{ tool }]`, the tools the agent actually called.

### ai-service itself — `/api/ai/*`

| Method | Path                         | Body                                                  | Returns                                                  |
|--------|------------------------------|-------------------------------------------------------|----------------------------------------------------------|
| POST   | <code>/api/ai/chat</code>    | <code>{ question, k?, context?, history? }</code>     | <code>{ answer, sources }</code>                         |
| POST   | <code>/api/ai/agent</code>   | <code>{ question, context, history? }</code>          | <code>{ answer, steps }</code>                           |
| POST   | <code>/api/ai/search</code>  | <code>{ query, k? }</code>                            | <code>{ results }</code> — retrieval only, no generation |
| POST   | <code>/api/ai/expand</code>  | <code>{ skill, subcategory?, count? }</code>          | <code>{ skills }</code>                                  |
| POST   | <code>/api/ai/narrate</code> | <code>{ role, readiness?, phases, strengths? }</code> | <code>{ narration, rationales, sources }</code>          |
| GET    | <code>/api/ai/health</code>  | —                                                     | <code>{ ok, service, chunks }</code>                     |

`context` is the student payload the Next routes assemble: `skills`, `edges`, `requirements`, `demonstrated`, `roles`, `target_role`, `recent_assessments` and `available_assessments`. `k` is how many chunks to retrieve, default 4.

`recent_assessments` is the last three completed sittings as `{ slug, title, score, maxScore, completedAt }`, the most recent one carrying a `weakest` array of `{ name, correct, total }` — the three worst skills of its breakdown. It is what lets `/chat` answer "how did I do?" from the graded result rather than from a skill level, which cannot tell a test from a self-reported claim. `/agent` is sent the same field but never renders it into the prompt — it reaches the data through `get_assessment_history`, so a conversation that never asks about a score never pays for one.

`available_assessments` is every assessment that currently exists, as `{ slug, title }` — not just ones this student has sat. `/chat` is the only consumer: it's what lets the system prompt link an assessment by slug without that list being hardcoded and going stale the next time one is added.

`/narrate` is called by skill-service, not by the browser, and it is the only endpoint whose output is written to the database. It receives a plan that has already been computed — phases, titles, order, week estimates — and returns prose about it: one narration and one rationale per phase. A rationale for a phase number that was not sent is dropped rather than stored, which is the one way a model could otherwise change the shape of a plan it was only asked to describe.

## The agent's tools

`/agent` runs a DeepSeek tool-calling loop. Six tools, four of them named by the problem statement:

| Tool                                   | Does                                                                             |
|----------------------------------------|----------------------------------------------------------------------------------|
| <code>analyze_student_skills</code>    | Current levels, per-category strength, readiness inputs                          |
| <code>generate_skill_gap</code>        | Calls python-analyzer <code>/gaps</code> — weighted gaps against the target role |
| <code>create_roadmap</code>            | Calls python-analyzer <code>/roadmap</code> — phases from the topological sort   |
| <code>compare_target_roles</code>      | Ranks every seeded role against what the student has                             |
| <code>search_learning_resources</code> | Seeded <code>resources</code> rows for the skills in question                    |
| <code>get_assessment_history</code>    | The student's graded sittings and the weakest skills of the latest               |

The model chooses which to call and in what order; it never computes a number itself. Readiness percentages, gap counts and phase ordering come from the analyzer or not at all — the system prompt forbids guessing them, because a model inventing a readiness score would contradict the dashboard.

## The python-analyzer contract

skill-service calls `POST {PYTHON_ANALYZER_URL}/roadmap` with a 4s timeout and falls back to a local topological layering on any failure. This is the shape it returns.

python-analyzer also serves `/analyze`, `/gaps`, `/score`, `/compare` and `/plan`. `/roadmap` speaks ai-service's `AnalysisRequest`; `/plan` speaks skill-service's `SkillRow`. The two shapes are deliberate — changing one does not change the other.

Request:

```
{
  "role": { "slug": "ai-engineer", "name": "AI Engineer" },
  "skills": [
    {
      "id": "uuid",
      "slug": "numpy",
      "name": "NumPy",
      "subcategory": "Data Analysis",
      "category": "Data and AI",
      "mastery": "gap",
      "level": 0,
      "requiredLevel": 4,
      "weight": 5,
      "prerequisites": ["uuid", "uuid"]
    }
  ]
}
```

Response:

```
{
  "phases": [
    {
      "phase": 1,
      "title": "Data Analysis and Databases",
      "rationale": null,
      "estimatedWeeks": 6,
      "skills": [{ "slug": "numpy", "ordinal": 0, "gapScore": 20 }]
    }
  ],
  "readinessScore": 15,
  "narration": null
}
```

`phase` is a rank of the topological sort: skills sharing one can be learned in parallel. Ordering is the analyzer's to decide and never a model's — `narration` and `rationale` are the only fields an LLM writes.

## Health

---

Every service answers `GET /health` with `{ ok: true, service }` and nothing sensitive. Compose and Kubernetes both probe it.